

Politechnika Świętokrzyska
Podstawy programowania

Sprawozdanie z realizacji zadań projektowych

Opis wykonania zadań na podstawie repozytorium projektu

Wykonały:

Maja Guzy (098285)

Weronika Duda (091829)

Sara Białkowska (098295)

Inżynieria Danych

II semestr, tryb stacjonarny

1. Wstęp i zakres sprawozdania

Celem projektu było praktyczne wykorzystanie wiedzy zdobytej w trakcie wykładów oraz laboratoriów prowadzonych w ramach przedmiotu Podstawy Programowania. W projekcie wykorzystano środowisko MSYS2 oraz narzędzia systemów GNU/Linux służące do przetwarzania danych, automatyzacji czynności, pracy z plikami tekstowymi, przetwarzania obrazów oraz generowania plików wynikowych.

Sprawozdanie zostało opracowane na podstawie zawartości repozytorium projektu zawartego w systemie GitHub. Analizie poddano przede wszystkim pliki skryptów, dokumentację roboczą oraz pliki wynikowe znajdujące się w repozytorium. Repozytorium można znaleźć pod linkiem: <https://github.com/Giyaaa/PodstawyProgramowania>.

Zadania 1 i 2 nie zostały opisane w osobnych rozdziałach, ponieważ miały one na celu przygotowanie do pracy oraz zapoznanie się ze środowiskiem. Dotyczyły one utworzenia zespołu, przygotowania repozytorium, zapoznania się z dokumentacją oraz instalacji i konfiguracji środowiska MSYS2. Dalsza część sprawozdania koncentruje się na zadaniach praktycznych, dla których w repozytorium znajdują się polecenia, skrypty lub pliki wynikowe.

2. Wykorzystane narzędzia

W projekcie wykorzystano przede wszystkim standardowe narzędzia konsolowe dostępne w środowiskach GNU/Linux. Ich zastosowanie umożliwiło automatyczne przetwarzanie plików bez konieczności ręcznej edycji danych.

awk - przetwarzanie danych tekstowych według zadanego wzorca oraz generowanie nowych linii wynikowych.

sed - wykonywanie prostych zamian tekstowych, m.in. usuwanie znaków końca linii CR oraz czyszczenie fragmentów tekstu.

grep - wyszukiwanie wybranych fragmentów linii, np. kluczy w plikach JSON5.

diff i **patch** - porównywanie plików oraz nakładanie przygotowanych poprawek.

paste - łączenie kolejnych wierszy danych w wiele kolumn.

md5sum - sprawdzanie zgodności plików przez porównanie sum kontrolnych.

zip i **unzip** - rozpakowywanie i pakowanie archiwów.

find, **cut**, **basename** oraz konstrukcje powłoki **Bash** - iteracja po plikach oraz wydzielanie elementów nazw.

ImageMagick, w tym polecenia **mogrify**, **magick** i **montage** - konwersja oraz przetwarzanie plików graficznych.

3. Zadania

3.1. Zadanie 3. *Niesforne dane*

W repozytorium zapisano polecenie, które w jednej instrukcji tworzy plik wynikowy. Najpierw za pomocą printf dopisywany jest nagłówek zawierający nazwy kolumn: x, y oraz z. Następnie dane z pliku wejściowego są przekazywane przez potok do dalszego przetwarzania.

Wykorzystanie sed pozwala usunąć znak CR, który może występować w plikach utworzonych w systemie Windows. Jest to istotne, ponieważ pozostawienie tego znaku mogłoby spowodować nieprawidłowe wyświetlanie danych lub problemy przy dalszym przetwarzaniu w środowisku MSYS2/GNU. Polecenie paste z trzema argumentami „-” pobiera kolejne trzy wiersze ze standardowego wejścia i łączy je w jeden wiersz wynikowy. Dzięki temu z danych zapisanych w jednej kolumnie powstaje tabela składająca się z trzech kolumn. Całość zostaje zapisana do pliku dane_kolumny.txt. Plik wynikowy znajdujący się w repozytorium zawiera nagłówek oraz dane rozdzielone tabulatorami. Pierwsze wiersze mają postać: x, y, z, a następnie kolejne trójki wartości liczbowych.

Rezultat

Utworzono plik dane_kolumny.txt zawierający dane ułożone w trzech kolumnach. Taki format nadaje się do importu do arkusza kalkulacyjnego, np. w celu przygotowania wykresów.

3.2. Zadanie 4. *Dodawanie poprawek*

Pierwszym etapem była konwersja plików tekstowych do formatu końców linii stosowanego w systemach Unix/Linux. W dokumentacji roboczej wskazano użycie programu dos2unix dla obu porównywanych plików. Dzięki temu porównanie dotyczyło rzeczywistych różnic w treści, a nie różnic wynikających z formatu zapisu końca linii.

Następnie użyto programu diff w wariantcie unified diff. Wynikiem działania polecenia był plik lista.patch. Plik ten zawiera zestaw zmian koniecznych do przekształcenia pierwotnej listy w wersję poprawioną. W repozytorium widoczna łąka dodaje między innymi słowa: dzik, jest, dziki, i, ma, pokręcony oraz ogon. Kolejnym krokiem było zastosowanie łąki za pomocą programu patch. Polecenie patch lista.txt < lista.patch wprowadza zmiany do pliku lista.txt na podstawie przygotowanej łąki.

Na końcu obliczono sumy kontrolne MD5 dla zaktualizowanego pliku lista.txt oraz pliku lista-pop.txt. W repozytorium oba pliki z sumami kontrolnymi zawierają tę samą wartość: 683c1c85343c7337adfb13acb7598237. Oznacza to, że po aktualizacji pliki są zgodne.

Rezultat

Przygotowano plik lista.patch oraz potwierdzono zgodność plików po aktualizacji przez porównanie sum kontrolnych MD5.

3.3. Zadanie 5. Z CSV do SQL i z powrotem

W dokumentacji roboczej repozytorium zapisano polecenie realizujące konwersję danych CSV do instrukcji SQL. Polecenie `tail -n +2` pomija pierwszy wiersz pliku CSV, czyli nagłówek. Następnie dane są przekazywane do `awk`. Program `awk` został uruchomiony z separatorem pól ustawionym na średnik. Dzięki opcji `-F';'` kolejne wartości z wiersza CSV są dostępne jako pola `$1`, `$2` oraz `$3`. Na ich podstawie generowana jest instrukcja `INSERT INTO stepsData (time, intensity, steps) VALUES (...)`.

Drugi etap konwersji rozpoczyna się od polecenia `printf` tworzącego nagłówek `dateTime;steps;synced` i zapisującego go do pliku `steps.csv`. Następnie polecenie `grep` wyciąga zawartość nawiasów z każdej instrukcji `INSERT`, a `awk` dzieli uzyskane wartości separatorem, na pola `$1`, `$2` oraz `$3`. Wartość pierwszego pola jest podzielna przez 1000, co wynika ze zmiany dokładności stempla czasu z milisekund na sekundy i konieczności usunięcia trzech ostatnich zer z wartości `dateTime`. Wynikowe wiersze są dopisywane do pliku `steps.csv`.

Rezultat

Wynikiem wykonania skryptu są dwa pliki `steps.sql` oraz `steps.csv`. Pierwszy zawiera instrukcje `INSERT` generowane na podstawie danych z `steps-2sql.csv`, gotowe do importu do bazy danych SQL. A drugi zawiera dane z `steps-2csv.sql` przekonwertowane z powrotem do formatu CSV z nagłówkiem i wartościami czasu przeliczonymi z milisekund na sekundy.

3.4. Zadanie 6. *Marudny tłumacz*

Pierwsza część skryptu tworzy plik `pl-7.2.json5` na podstawie wersji angielskiej `en-7.2.json5`. Użyto programu `awk`, który dla każdej linii rozpoczynającej się od dwóch spacji i cudzysłowu wypisuje najpierw kopię tej linii poprzedzoną znakami komentarza `//`, a następnie oryginalną linię. W ten sposób każda fraza otrzymuje wersję komentarzową oraz aktywny wpis przeznaczony do tłumaczenia.

Drugim etapem jest porównanie kluczy występujących w plikach `en-7.2.json5` oraz `en-7.4.json5`. Program `grep` wyodrębnia fragmenty wyglądające jak klucze JSON5, sed usuwa dwukropki i cudzysłowy, a `sort` porządkuje klucze alfabetycznie. Sortowanie jest konieczne, ponieważ program `comm` wymaga uporządkowanych plików wejściowych. Polecenie `comm -13` wskazuje klucze występujące w drugiej liście, których nie ma w pierwszej. Otrzymana lista nowych kluczy zapisywana jest do pliku tymczasowego `/tmp/new_keys.txt`.

Na podstawie nowych kluczy skrypt tworzy plik `pl-7.4.json5`. Najpierw zapisuje znak otwierający obiekt, następnie w pętli `while` odczytuje kolejne klucze i wyszukuje odpowiadające im linie w pliku `en-7.4.json5`. Na końcu dopisywany jest znak zamykający. Zgodnie z plikiem wynikowym w repozytorium `pl-7.4.json5` zawiera 303 nowe aktywne wpisy.

Rezultat

Wynikiem wykonania skryptu są dwa pliki: `pl-7.2.json5` z liniami zdublowanymi w formie komentarza i oryginalnego wpisu oraz `pl-7.4.json5` zawierający wyłącznie nowe frazy z wersji 7.4.

3.5. Zadanie 7. *Fotografik gamoń*

Skrypt rozpoczyna działanie od rozpakowania archiwów kopie-1.zip oraz kopie-2.zip. Użyto opcji -o, aby nadpisywać istniejące pliki, oraz -q, aby ograniczyć liczbę komunikatów wypisywanych na ekran. Następnie wykonano pętlę for po wszystkich plikach ZIP znajdujących się w bieżącym katalogu. Każde archiwum jest rozpakowywane. Pozwala to obsłużyć sytuację, w której archiwa główne zawierają kolejne archiwa ze zdjęciami.

Właściwa obróbka plików graficznych wykonywana jest poleceniem mogrify. Zastosowano ustawienie jednostek PixelsPerInch oraz gęstość 96 DPI. Opcja -resize x720 zmienia wysokość obrazu na 720 pikseli, zachowując proporcje szerokości. Opcja -format jpg powoduje zapis wyniku w formacie JPG. Po przetworzeniu obrazów skrypt pakuje wszystkie pliki JPG do archiwum wynik.zip. Następnie wykonuje przeniesienie archiwum do ścieżki ./wynik oraz usuwa robocze pliki JPG, PNG i ZIP z bieżącego katalogu.

Rezultat

Skrypt automatyzuje proces rozpakowania, konwersji i zmniejszenia zdjęć oraz przygotowuje archiwum wynikowe zawierające obrazy w formacie JPG.

3.6. Zadanie 8. *Wszędzie te PDF-y*

Skrypt Zadanie8.sh rozpoczyna się od ustawienia set -e. Oznacza to, że w razie wystąpienia błędu skrypt powinien zakończyć działanie, co ogranicza ryzyko kontynuowania pracy na niepełnych danych. Pierwszym etapem jest rozpakowanie archiwów kopie-1.zip i kopie-2.zip oraz rozpakowanie pozostałych plików ZIP znajdujących się w katalogu roboczym. Następnie wykonywana jest konwersja obrazów. Polecenie mogrify ustawia rozdzielczość 96 DPI, zmienia wysokość obrazów na 720 pikseli oraz zapisuje wynik w formacie JPG.

Kolejny etap polega na przygotowaniu podpisów pod zdjęciami. Skrypt tworzy katalog podpisy i dla każdego pliku JPG uruchamia polecenie magick. Operacja dodaje biały pasek pod obrazem oraz umieszcza na nim nazwę pliku. Podpisany obraz zapisywany jest w katalogu podpisy. Następnie skrypt buduje strony portfolio. Tablica files zawiera listę podpisanych zdjęć. Pętla while pobiera po osiem plików, a polecenie magick montage układa je w siatce 2x4. Ustawienia -tile 2x4 oraz -geometry 480x360+30+30 odpowiadają za liczbę obrazów w wierszu i kolumnie oraz za odstępy między nimi.

Ostatnim etapem jest połączenie wygenerowanych stron PNG w dokument portfolio.pdf. Skrypt ustawia stronę A4 oraz tło białe. Dodatkowo tworzone jest archiwum wynik.zip zawierające pliki JPG, przenoszone następnie do katalogu wynik.

Rezultat

W wyniku działania skryptu tworzony jest dokument portfolio.pdf z ułożonymi zdjęciami i podpisami oraz archiwum wynik.zip zawierające przetworzone obrazy.

3.7. Zadanie 9. *Porządki w kopiach zapasowych*

Skrypt najpierw rozpakowuje dwa archiwa główne. Następnie tworzy katalog kopie, w którym budowana jest docelowa struktura katalogów. W pętli for przeglądane są wszystkie pliki ZIP znajdujące się w katalogu roboczym. Skrypt pomija archiwa kopie-1.zip i kopie-2.zip, aby nie przenosić archiwów źródłowych do struktury wynikowej.

Dla każdego pozostałego pliku ZIP wyodrębniany jest rok oraz miesiąc. Wykorzystano do tego polecenie `cut` z separatorem „-”. Dla nazwy w formacie RRRR-MM-DD.zip pierwszy element odpowiada rokowi, a drugi miesiącowi. Następnie tworzony jest katalog `kopie/rok/miesiąc`, a plik archiwum przenoszony jest do właściwego podkatalogu. Dzięki użyciu `mkdir -p` skrypt nie zgłasza błędu, jeśli katalog już istnieje.

Rezultat

Po wykonaniu skryptu pliki ZIP z kopiami zapasowymi zostają uporządkowane w katalogach według roku i miesiąca.

3.8. Zadanie 10. *Galeria dla grafika*

Skrypt definiuje na początku zmienne opisujące nazwy najważniejszych elementów: archiwum wejściowego `wynik.zip`, katalogu tymczasowego `wynik`, katalogu galerii oraz pliku `index.html`. Dzięki temu dalsza część skryptu jest czytelniejsza.

Archiwum `wynik.zip` jest rozpakowywane do katalogu `wynik`. Następnie tworzony jest katalog `galeria`, do którego kopiowane są wszystkie pliki JPG. Plik HTML powstaje w katalogu galerii. Pierwsza część dokumentu HTML jest zapisywana przy użyciu konstrukcji `here document`. Zawiera ona nagłówek, style CSS, tytuł strony oraz początek sekcji `body`. Style odpowiadają za układ responsywny i wygląd kafelków galerii.

Następnie skrypt wykonuje pętlę po wszystkich plikach JPG znajdujących się w katalogu galerii. Dla każdego pliku pobierana jest sama nazwa pliku za pomocą `basename`. Na tej podstawie do pliku `index.html` dopisywany jest blok HTML zawierający odnośnik do pliku, znacznik `img` oraz podpis z nazwą zdjęcia.

Po przetworzeniu wszystkich obrazów skrypt dopisuje końcową część dokumentu HTML. W repozytorium znajduje się wygenerowany plik `index.html` zawierający 269 bloków galerii, co wskazuje na automatyczne tworzenie wpisów dla wielu zdjęć.

Rezultat

Utworzono galerię internetową w postaci pliku HTML oraz zestawu obrazów JPG. Każdy obraz otrzymuje własny blok HTML i podpis odpowiadający nazwie pliku.

4. Podsumowanie

Projekt obejmował zestaw zadań pokazujących praktyczne zastosowanie narzędzi konsolowych do przetwarzania danych tekstowych, porównywania plików, automatyzacji konwersji obrazów oraz generowania plików wynikowych. Najczęściej wykorzystywanym mechanizmem był potok, który pozwalał przekazywać wynik jednego polecenia jako dane wejściowe kolejnego programu.

W zadaniach tekstowych wykorzystano przede wszystkim `awk`, `sed`, `grep`, `paste`, `diff`, `patch` oraz `md5sum`. Narzędzia te pozwoliły przekształcać dane między formatami, przygotować łańkę aktualizującą plik oraz sprawdzać zgodność wyników. W zadaniach dotyczących obrazów zastosowano `ImageMagick`, który umożliwił masową zmianę rozmiaru, konwersję formatu, dodawanie podpisów oraz tworzenie portfolio PDF.

Zadania 7-10 szczególnie dobrze pokazują sens stosowania skryptów powłoki. Operacje wykonywane ręcznie dla kilkudziesięciu lub kilkuset plików byłyby czasochłonne i podatne na błędy. Dzięki pętlom oraz zmiennym w Bashu możliwe było wykonanie ich w sposób automatyczny i powtarzalny.